SCFC · v3.1 — Podcast Supplement

> **Document type**: Additive supplement to `scfc-masters-spec-v3.md`. Adds podcast/audio functionality. **Pre-requisite**: v3 spec is the base. This document only describes what changes for podcasts. **Scope**: 80 audio episodes (one per day), pre-generated MP3s using xAI Grok TTS API, two-voice host/expert format, served as static files. New audio player UI integrated into the Machine.

---

0 · Why this is a supplement, not a rewrite

Podcasts add a content layer (80 scripts × ~750 words = ~60,000 words of new authored content) and a runtime UI feature (audio player + playback persistence). Neither requires architectural changes — the engine still has no backend and runs as static files.

This document specifies:

1. The build pipeline for generating MP3s (§2)
2. The script schema and authoring approach (§3)
3. The runtime audio player UI (§4)
4. State additions for playback position (§5)
5. Acceptance criteria for the audio feature (§6)

It does not respec the curriculum, design system, or persistence model from v2/v3. Those remain in force.

---

1 · Decisions locked

Question	Decision
TTS provider	**xAI Grok TTS** (`POST https://api.x.ai/v1/tts`)
Audio serving	**Pre-generated** at build time, served as static MP3 files
Episode length	**~5 minutes** per day (target 700–850 words per script)

Voice character	**Two voices** alternating — Host + Expert dialogue
Host voice	`rex` — confident, clear, professional (per xAI docs guidance for tutorials)
Expert voice	`ara` — warm, friendly, conversational
Output format	MP3 at 24 kHz / 128 kbps (Grok TTS default)
Total content	80 episodes × ~5 min = ~400 minutes of audio
Total file size	~80 MB across all 80 episodes (avg ~1 MB / episode)
Total TTS cost	~$5–8 (60,000 words × ~5 chars/word ÷ 1M × $4.20)

---

2 · Build pipeline

The build pipeline is a one-time (or periodic, for content updates) script that generates 80 MP3 files. It runs locally; the agent runs it once, commits the resulting `audio/` directory, and the Machine plays static files thereafter.

2.1 Directory layout

```
/dist/
  index.html            (the Machine, v3 + v3.1 audio player)
  app.css               (if split build)
  app.js                (if split build)
  data.js               (if split build — includes audio metadata)
  audio/
    day-01.mp3          (Day 1 episode, ~1 MB)
    day-02.mp3
    ...
    day-80.mp3
  scripts/              (source for the build pipeline; can be excluded from prod
    day-01.txt
    day-02.txt
    ...
    day-80.txt
  build/
    generate-audio.js   (the build script)
    package.json
```

The `audio/` directory is shipped to production. The `scripts/` and `build/` directories are source-of-truth for re-generation but not required at runtime.

2.2 Script source format (`scripts/day-NN.txt`)

Each script file is plain text with two-line speaker tags. Format:

```
[REX]
Welcome back to SCFC. Today is day eleven, and we're talking about
something deceptively simple — the Economic Order Quantity formula.
[pause]
[ARA]
The formula itself is from 1913. Ford Whitman Harris published it in a
trade magazine called Factory. Twelve pages. No fanfare. And it's still
the answer to one of the most asked questions in inventory management.
[REX]
So why are we still talking about it more than a century later?
[ARA]
[laugh] Because most people get the lesson wrong. They think the lesson
is the formula. The lesson is actually that the cost curve around the
optimum is flat. <emphasis>Flat.</emphasis> That changes everything.
...
```

Rules:

- Speaker tag is `[REX]` or `[ARA]` on its own line, before the speaker's lines.
- Inline speech tags from the Grok TTS API spec are allowed: `[pause]`, `[long-pause]`, `[laugh]`, `[chuckle]`, `[sigh]`, `[breath]`.
- Wrapping tags allowed: `<emphasis>...</emphasis>`, `<whisper>...</whisper>`, `<slow>...</slow>`.
- Each speaker block is sent as a separate TTS API call with the corresponding voice. The build script concatenates the resulting audio buffers into a single MP3.

2.3 The build script (`build/generate-audio.js`)

Node.js, runs locally. Reads each `scripts/day-NN.txt`, parses speaker blocks, calls Grok TTS API for each block, concatenates the audio.

```
// build/generate-audio.js
import fs from 'fs';
import path from 'path';

const XAI_API_KEY = process.env.XAI_API_KEY;
if (!XAI_API_KEY) { console.error('Set XAI_API_KEY env var.'); process.exit(1); }

const SCRIPTS_DIR = path.join(__dirname, '..', 'scripts');
```

```
const OUT_DIR = path.join(__dirname, '..', 'dist', 'audio');
fs.mkdirSync(OUT_DIR, { recursive: true });

const VOICES = { REX: 'rex', ARA: 'ara' };

async function ttsCall(text, voice) {
  // Retry with backoff on 429/500/503 (per xAI docs)
  for (let attempt = 0; attempt < 4; attempt++) {
    const res = await fetch('https://api.x.ai/v1/tts', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${XAI_API_KEY}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        text,
        voice_id: voice,
        language: 'en',
        output_format: { codec: 'mp3', sample_rate: 24000, bit_rate: 128000 }
      }),
    });
    if (res.ok) return Buffer.from(await res.arrayBuffer());
    if ([429, 500, 503].includes(res.status)) {
      await new Promise(r => setTimeout(r, Math.pow(2, attempt) * 1000));
      continue;
    }
    throw new Error(`TTS error ${res.status}: ${await res.text()}`);
  }
  throw new Error('Max retries exceeded');
}

function parseScript(content) {
  // Split on speaker tags. Returns [{voice, text}, ...]
  const lines = content.split('\n');
  const blocks = [];
  let currentSpeaker = null;
  let currentText = [];
  for (const line of lines) {
    const m = line.trim().match(/^\[(REX|ARA)\]$/);
    if (m) {
      if (currentSpeaker && currentText.length) {
        blocks.push({ voice: VOICES[currentSpeaker], text: currentText.join(' ').
      }
      currentSpeaker = m[1];
      currentText = [];
    } else if (currentSpeaker) {
      currentText.push(line);
```

```
    }
  }
  if (currentSpeaker && currentText.length) {
    blocks.push({ voice: VOICES[currentSpeaker], text: currentText.join(' ').trim
  }
  return blocks.filter(b => b.text);
}

async function generateDay(scriptPath, outputPath) {
  console.log(`-> generating ${path.basename(outputPath)}`);
  const content = fs.readFileSync(scriptPath, 'utf-8');
  const blocks = parseScript(content);
  const audioBuffers = [];
  for (const block of blocks) {
    const audio = await ttsCall(block.text, block.voice);
    audioBuffers.push(audio);
    // Insert a 200ms silence buffer between speaker turns (~9.6 KB at 24kHz/128k
    // Simplest: skip — the TTS audio already has natural beat-ends and Grok's vo
  }
  const combined = Buffer.concat(audioBuffers);
  fs.writeFileSync(outputPath, combined);
  console.log(`   ${(combined.length / 1024).toFixed(1)} KB written.`);
}

async function main() {
  const scripts = fs.readdirSync(SCRIPTS_DIR).filter(f => f.match(/^day-\d{2}\.tx
  for (const scriptFile of scripts) {
    const dayNum = scriptFile.match(/day-(\d{2})/)[1];
    const outputPath = path.join(OUT_DIR, `day-${dayNum}.mp3`);
    if (fs.existsSync(outputPath) && !process.env.FORCE_REGEN) {
      console.log(`-- skip day-${dayNum}.mp3 (exists; set FORCE_REGEN=1 to regene
      continue;
    }
    await generateDay(path.join(SCRIPTS_DIR, scriptFile), outputPath);
    await new Promise(r => setTimeout(r, 500)); // gentle rate limiting
  }
  console.log('done.');
}

main().catch(e => { console.error(e); process.exit(1); });
```

2.4 MP3 concatenation note

Concatenating raw MP3 frame-bytes (as the script above does with `Buffer.concat`) **works** for sequential playback in HTML5 `<audio>` elements and in browsers. MP3 is a frame-based format and most decoders tolerate concatenated frames from the same encoder.

If duration metadata becomes inaccurate (some browsers compute duration from header alone, not actual content), the build script can post-process with `ffmpeg`:

```
ffmpeg -y -i "concat:block1.mp3|block2.mp3|block3.mp3" -c copy day-NN.mp3
```

Or, more robustly, decode and re-encode through ffmpeg in a single step. For v3.1, the simple concat approach is acceptable; if duration display breaks, fall back to ffmpeg.

2.5 Build script execution

```
cd build
npm install
export XAI_API_KEY="xai-..."
node generate-audio.js
# Output: 80 MP3 files in dist/audio/
```

The script is **idempotent**: existing MP3 files are skipped. To regenerate, set `FORCE_REGEN=1`. To regenerate a specific day, delete that file and re-run.

---

3 · Script authoring

3.1 Script structure (per episode)

Each ~5-minute script follows a consistent template — the same pacing the user gets used to across 80 episodes.

```
[REX] · COLD OPEN (~30 sec)
  Day number, framework name, why this framework matters in one sentence.

[ARA] · ORIGINATOR & CONTEXT (~45 sec)
  The historical figure, the year, the field problem they were solving.
  Why the framework outlived its original context.

[REX] · THE INTUITION (~60 sec)
  The plain-English version. What the framework actually says, no jargon.

[ARA] · THE MECHANICS (~90 sec)
  How it works in practice. If math is involved, the math is described in
  words ("the formula multiplies demand by ordering cost, divides by
  holding cost, takes the square root"). For listeners, not for derivation.

[REX] · THE FAILURE MODE (~45 sec)
```

```
  How people get this wrong. The most common misapplication. The senior
  insight that separates a user from a practitioner.

[ARA] · THE PRINCIPAL'S MOVE (~30 sec)
  What a senior operator does differently. One concrete behavioral
  prescription.

[REX] · CLOSE (~20 sec)
  Tomorrow's day teaser, one sentence.
```

Total target: 700–850 words. Spoken at ~150 wpm (Grok TTS default), this lands at 4:40–5:40 minutes.

3.2 Authoring approach for the agent

The agent does not need to write 80 scripts manually. The recommended pipeline:

1. For each curriculum day, the agent has access to:
   - Day title (`t`)
   - Framework name (`f`)
   - Originator + year (`o`)
   - Citation (`cite`)
   - Summary (`sum`)
   - Principal framing (`fr` — for case days)
2. The agent uses an LLM (any capable text model — GPT-5.5, Claude, etc.) with the script template above to draft each episode.
3. Drafts are saved to `scripts/day-NN.txt` for human review.
4. After review, the build script generates audio.

Authoring prompt template (for the LLM that writes the scripts):

```
You are writing a 5-minute audio script for a supply chain education
podcast called SCFC. The episode covers the following curriculum day:

DAY: {d}
TITLE: {t}
FRAMEWORK: {f}
ORIGINATOR: {o}
CITATION: {cite}
```

```
SUMMARY: {sum}
PRINCIPAL FRAMING: {fr}

Write a two-voice script following this structure:
- [REX] cold open (~30 sec)
- [ARA] originator and context (~45 sec)
- [REX] the intuition (~60 sec)
- [ARA] the mechanics (~90 sec)
- [REX] the failure mode (~45 sec)
- [ARA] the principal's move (~30 sec)
- [REX] close, with one-sentence teaser for day {d+1} ({next_day_title})

Total length: 700-850 words, spoken at ~150 wpm.

Voices:
- REX is the confident, clear host. Asks questions, frames context.
- ARA is the warm, knowledgeable expert. Provides depth and examples.

Tone: NPR-style. Substantive but conversational. No jargon without
explanation. No corporate-speak. Treat the listener as a smart
professional who is new to this specific framework.

Use inline speech tags sparingly:
- [pause] for dramatic timing
- [laugh] or [chuckle] only where genuinely warranted
- <emphasis>word</emphasis> for the most important term in a sentence

Output the script in plain text with [REX] / [ARA] tags on their own
lines. Do not include any commentary or metadata.
```

3.3 Script review checklist

For each draft, the human reviewer checks:

- Word count is 700–850.
- All key concepts from the curriculum day are mentioned.
- Citations are accurate (year, name, journal).
- Tone matches NPR-style — substantive but conversational.
- Speech tags are used sparingly and naturally.
- No factual errors (especially historical dates and citation details).
- The “failure mode” section actually identifies the common error, not platitudes.

- The “principal’s move” is a concrete behavioral prescription.

---

4 · Audio player UI

The audio player integrates into the Machine without breaking the existing aesthetic. Three components:

4.1 Player location

The audio player lives in the **lower section** of the panel, between the controls row and the primary button. It only appears on `home` view (today’s lecture) and on `curr` view when a day is selected.

Visually, it reads as a **cassette deck slot** — narrow, mechanical, with the Machine’s putty/aluminum gradient.

4.2 Player CSS

```
.audio-deck {
  background: #B0A892;
  border-radius: 6px;
  padding: 14px 16px;
  margin-bottom: 14px;
  position: relative;
  z-index: 1;
  box-shadow:
    inset 0 1px 0 rgba(255,255,255,0.5),
    inset 0 -1px 0 rgba(0,0,0,0.2),
    0 0 0 1px rgba(0,0,0,0.14);
  display: grid;
  grid-template-columns: auto 1fr auto;
  gap: 14px;
  align-items: center;
}
.audio-deck-label {
  font-family: 'IBM Plex Mono', monospace;
  font-size: 9px;
  letter-spacing: 0.28em;
  text-transform: uppercase;
  color: #5C5446;
}
.audio-deck-label::before { content: '▶ '; color: #1A1814; }

.audio-deck-play {
```

```
  width: 44px; height: 44px; border-radius: 50%;
  background: linear-gradient(180deg, #D4CCB6 0%, #BFB7A1 100%);
  border: none; cursor: pointer;
  box-shadow:
    inset 0 1px 0 rgba(255,255,255,0.6),
    inset 0 -2px 0 rgba(0,0,0,0.22),
    0 1px 2px rgba(0,0,0,0.18),
    0 0 0 1px rgba(0,0,0,0.12);
  display: flex; align-items: center; justify-content: center;
  transition: transform 80ms;
}
.audio-deck-play:active { transform: translateY(2px); }
.audio-deck-play .icon {
  width: 0; height: 0;
  border-left: 12px solid #1A1814;
  border-top: 8px solid transparent;
  border-bottom: 8px solid transparent;
  margin-left: 3px;
}
.audio-deck-play.playing .icon {
  width: 12px; height: 14px;
  border: none;
  background:
    linear-gradient(to right,
      #1A1814 0%, #1A1814 35%,
      transparent 35%, transparent 65%,
      #1A1814 65%, #1A1814 100%);
  margin-left: 0;
}

.audio-deck-progress {
  flex: 1;
  height: 6px;
  background: rgba(0,0,0,0.25);
  border-radius: 1px;
  position: relative;
  cursor: pointer;
}
.audio-deck-progress-fill {
  position: absolute; top: 0; left: 0;
  height: 100%;
  background: linear-gradient(90deg, #C8442A 0%, #FFC080 100%);
  width: 0%;
  border-radius: 1px;
  transition: width 80ms linear;
}
.audio-deck-progress-handle {
```

```
  position: absolute; top: 50%;
  width: 12px; height: 12px;
  background: #1A1814; border: 2px solid #FFC080;
  border-radius: 50%;
  transform: translate(-50%, -50%);
  pointer-events: none;
  box-shadow: 0 0 4px rgba(255,192,128,0.7);
}

.audio-deck-time {
  font-family: 'Major Mono Display', monospace;
  font-size: 11px;
  color: #1A1814;
  letter-spacing: 0.06em;
  min-width: 76px;
  text-align: right;
}

.audio-deck.disabled .audio-deck-play {
  opacity: 0.4; cursor: not-allowed;
}
.audio-deck.disabled .audio-deck-progress { opacity: 0.4; }
.audio-deck.disabled .audio-deck-time { opacity: 0.4; }

@media (max-width: 720px) {
  .audio-deck { grid-template-columns: auto 1fr; }
  .audio-deck-label { display: none; }
  .audio-deck-time {
    grid-column: 1 / -1;
    text-align: left;
    margin-top: 4px;
    font-size: 10px;
  }
}
```

4.3 Player HTML (template)

```
<div class="audio-deck" id="audioDeck">
  <span class="audio-deck-label">EPISODE</span>
  <button class="audio-deck-play" id="audioPlay" aria-label="Play episode">
    <span class="icon"></span>
  </button>
  <div class="audio-deck-progress" id="audioProgress">
    <div class="audio-deck-progress-fill" id="audioProgressFill"></div>
    <div class="audio-deck-progress-handle" id="audioProgressHandle" style="left:
  </div>
```

```
  <div class="audio-deck-time" id="audioTime">0:00 / 0:00</div>
  <audio id="audioElement" preload="metadata"></audio>
</div>
```

4.4 Player JS

```
const audioElement = document.getElementById('audioElement');
const audioPlay = document.getElementById('audioPlay');
const audioProgress = document.getElementById('audioProgress');
const audioProgressFill = document.getElementById('audioProgressFill');
const audioProgressHandle = document.getElementById('audioProgressHandle');
const audioTime = document.getElementById('audioTime');
const audioDeck = document.getElementById('audioDeck');

function formatTime(seconds) {
  if (!isFinite(seconds)) return '0:00';
  const m = Math.floor(seconds / 60);
  const s = Math.floor(seconds % 60);
  return `${m}:${s.toString().padStart(2, '0')}`;
}

function loadEpisode(dayNum) {
  const path = `audio/day-${String(dayNum).padStart(2, '0')}.mp3`;
  audioElement.src = path;
  audioElement.load();
  audioDeck.classList.remove('disabled');

  // Restore last playback position from state
  const savedPos = state.audioPositions?.[dayNum] || 0;
  audioElement.addEventListener('loadedmetadata', () => {
    if (savedPos > 0 && savedPos < audioElement.duration - 5) {
      audioElement.currentTime = savedPos;
    }
    updateAudioUI();
  }, { once: true });
}

function updateAudioUI() {
  const cur = audioElement.currentTime || 0;
  const dur = audioElement.duration || 0;
  const pct = dur > 0 ? (cur / dur) * 100 : 0;
  audioProgressFill.style.width = pct + '%';
  audioProgressHandle.style.left = pct + '%';
  audioTime.textContent = `${formatTime(cur)} / ${formatTime(dur)}`;
  audioPlay.classList.toggle('playing', !audioElement.paused);
}
```

```
audioPlay.addEventListener('click', () => {
  if (audioElement.paused) audioElement.play();
  else audioElement.pause();
  updateAudioUI();
});

audioElement.addEventListener('play', updateAudioUI);
audioElement.addEventListener('pause', updateAudioUI);
audioElement.addEventListener('timeupdate', () => {
  updateAudioUI();
  // Persist playback position every 5 seconds
  if (Math.floor(audioElement.currentTime) % 5 === 0) {
    saveAudioPosition(state.currentDay, audioElement.currentTime);
  }
});
audioElement.addEventListener('ended', () => {
  saveAudioPosition(state.currentDay, 0); // reset on completion
  updateAudioUI();
  // Optional: mark "audio complete" as a separate completion event (see §5.3)
});

audioProgress.addEventListener('click', (e) => {
  const rect = audioProgress.getBoundingClientRect();
  const pct = (e.clientX - rect.left) / rect.width;
  audioElement.currentTime = pct * audioElement.duration;
  updateAudioUI();
});

function saveAudioPosition(dayNum, position) {
  if (!state.audioPositions) state.audioPositions = {};
  state.audioPositions[dayNum] = Math.floor(position);
  saveState();
}
```

4.5 When the player appears

The audio deck is visible:

- On `home` view, always — for the current day's episode.
- On `curr` view, when a specific day is "loaded" via `loadDay(d)`.

The audio deck is **hidden** on:

- `glos`, `read`, and palette views.

When the user switches views or loads a different day, the player auto-pauses and `loadEpisode(newDayNum)` is called.

4.6 Keyboard shortcut

Add to v2 §10.1 keyboard map:

Key	Action
`Space`	Toggle play/pause (when audio deck is visible and not in input focus)

```
document.addEventListener('keydown', (e) => {
  if (e.key === ' ' && !e.target.matches('input, textarea')) {
    if (audioDeck && !audioDeck.classList.contains('disabled')) {
      e.preventDefault();
      audioPlay.click();
    }
  }
});
```

---

5 · State additions

5.1 Schema extension

Add to v3 state schema:

```
type State = {
  // ... existing v3 fields
  audioPositions?: Record<number, number>;  // dayNum → seconds played (last posi
  audioCompleted?: number[];                 // array of dayNums whose audio was
};
```

These are **optional** fields. A user without audio data still loads correctly.

5.2 Migration

The migration from v3 to v3.1 is trivial — both new fields default to undefined / empty. No data transformation needed.

5.3 Audio completion as a separate signal (optional)

Audio completion (reaching the end of an episode) is **not** the same as day completion

(engaging the case or marking the lecture complete). The user can:

- Listen to the podcast without engaging the case.
- Engage the case without listening to the podcast.
- Do both.

`audioCompleted` is tracked separately. It is surfaced in the curriculum grid as a small headphones icon (♪) on day cards where the audio has been completed. This gives the user a sense of audio progression independent of curriculum progression.

```
.curr-day .audio-mark {
  display: inline-block;
  font-size: 8px;
  color: #FFC080;
  margin-left: 4px;
}
```

```
<div class="dnum">D11 · CASE <span class="audio-mark">♪</span></div>
```

---

6 · Acceptance criteria (v3.1 additions)

Add to v3 acceptance checklist:

6.1 Build pipeline

- `build/generate-audio.js` runs without error given `XAI_API_KEY` env var.
- All 80 `scripts/day-NN.txt` files exist and parse correctly.
- All 80 `audio/day-NN.mp3` files are generated.
- Each MP3 plays in a browser (Chrome, Safari, mobile Safari) without errors.
- Each MP3 has duration in 4:30–5:40 range.
- Total audio output is < 100 MB.

6.2 Player UI

- Audio deck renders below the controls row on `home` and `curr` views.
- Audio deck is hidden on `glos` , `read` , and palette views.

- Play button has play/pause states with the correct icon swap.
- Progress bar fills as audio plays.
- Clicking the progress bar seeks correctly.
- Time display shows `M:SS / M:SS` format.
- Player works on mobile (375px viewport tested).
- Spacebar toggles play/pause when not in an input.

6.3 Persistence

- Pausing and reloading the page resumes from the same playback position (within 5 seconds).
- Switching to a different day loads that day's audio without auto-playing.
- Completing an episode marks `audioCompleted` for that day.
- Curriculum grid shows the ♪ mark on days with completed audio.

6.4 Voice quality (manual review)

- All 80 episodes audibly use both REX (host) and ARA (expert) voices.
- No episode contains audible artifacts at speaker transitions.
- No episode contains the literal speech tags being read aloud (e.g., "open square bracket pause close square bracket").
- Volume levels are consistent across episodes (within ~2 dB).

6.5 Failure modes

- If an MP3 file is missing, the player shows "EPISODE UNAVAILABLE" without crashing.
- If the user is offline and the MP3 isn't cached, a graceful error message appears.
- If TTS API generation fails for one day during build, the script continues and reports the failure.

---

7 · Cost & operational notes

7.1 One-time generation cost

- 80 episodes × ~750 words × ~5 chars/word = ~300,000 characters
- xAI TTS pricing: $4.20 per 1M characters
- **Total cost: ~$1.26**

Add ~3× margin for re-generation passes during script iteration: budget **$5–10** for the full build cycle.

7.2 Re-generation workflow

To regenerate a single episode (e.g., after editing day 47's script):

```
rm dist/audio/day-47.mp3
node build/generate-audio.js
```

To regenerate all episodes:

```
FORCE_REGEN=1 node build/generate-audio.js
```

7.3 No CDN required

80 MP3s × ~1 MB = ~80 MB total. Static hosting on Vercel, Netlify, GitHub Pages, or Cloudflare Pages handles this with no special configuration. The `audio/` directory ships with the rest of the build artifacts.

7.4 Future: voice cloning

xAI's Custom Voices feature (launched late April 2026) allows recording a 60-second voice sample and creating a `voice_id` usable in the same TTS endpoint. If the user (Kyle) wants the host voice to be their own — e.g., for personal branding or because they prefer their own voice — they can:

1. Record a verification phrase via the xAI console.
2. Receive a `voice_id` (e.g., `nlbqfwie`).
3. Replace `'rex'` with the custom `voice_id` in the build script.
4. Regenerate all 80 episodes (~$5).

This is out of scope for v3.1 but trivial to add later.

---

8 · Out of scope for v3.1

- Real-time TTS generation (i.e., generating audio on-demand from arbitrary scripts). All audio is pre-generated.
- Closed captions / transcripts displayed in sync with audio. The script files exist and could be used for this in v3.2; not built for v3.1.
- Variable playback speed (1.5×, 2×). Native `<audio>` elements support `playbackRate`, but no UI for it in v3.1.
- Skip-forward/skip-back buttons. Use the progress bar for now.
- Episode chaptering (jumping to "the mechanics" section within an episode). Not authored.
- Background playback when the app is in another tab. Browsers handle this differently; no special code.
- Download episodes for offline listening as a UI affordance. Files are already cached by the browser after first play.

---

9 · Definition of done — v3.1

The v3.1 build is done when:

1. All v3 acceptance criteria pass.
2. Every item in §6 above passes.
3. `audio/day-01.mp3` through `audio/day-80.mp3` exist as static files in the production deploy.
4. A user opens the app, sees the audio deck on the home view, presses play, and hears a coherent two-voice ~5-minute episode about the day's framework.
5. The user can pause, scrub, navigate to another day, return, and resume from where they left off.
6. The audio is high-enough quality that a senior procurement leader (Kyle) would listen to it during a commute without irritation.

That is the bar. Build to it.